ロール管理API設計書

メタデータ

- **作成日**: 2026-02-13
- **バージョン**: 1.0.0
- **最終更新**: 2026-02-13
- **担当**: Development Team
- **実装状況**: ✅ 実装済み（100%）

---

1. ロール一覧取得API

概要

組織内のロール一覧と各ロールに紐づく権限情報を取得します。検索、フィルタリング機能に対応します。

エンドポイント

```
GET /api/v1/master/role
```

HTTPメソッド

GET

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.get('/role',
  authMiddleware,              // JWT認証（organizationIdを取得）
  roleController.getRoles
);
```

リクエスト

クエリパラメータ

現在はクエリパラメータなしで全件取得します。（将来的に検索・フィルタリング機能を追加予定）

リクエスト例

```
GET /api/v1/master/role
```

レスポンス

成功時（200 OK）


```
{
  "success": true,
  "data": [
    {
      "roleId": "system_admin",
      "roleName": "システム管理者",
      "functionName": "user_management,master_management",
      "initialScreenFunctionName": "system_overview",
      "isActive": true,
      "functions": [
        {
          "name": "user_management",
          "icon": "Ⓦ"
        },
        {
          "name": "master_management",
```

```
          "icon": "Ⓡ"
        }
      ],
      "updatedAt": "2026-02-13T10:00:00Z"
    },
    {
      "roleId": "data_admin",
      "roleName": "データ管理者",
      "functionName": "master_management",
      "initialScreenFunctionName": "master_management",
      "isActive": true,
      "functions": [
        {
          "name": "master_management",
          "icon": "Ⓡ"
        }
      ],
      "updatedAt": "2026-02-13T10:00:00Z"
    }
  ],
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```


レスポンスフィールド説明:

- `roleId` : ロールCD（一意の識別子）
- `roleName` : ロール名
- `functionName` : 権限を持つ機能名（カンマ区切り）
- `initialScreenFunctionName` : 初期画面機能名
- `isActive` : アクティブ状態
- `functions` : 各機能の詳細情報（配列）
  - `name` : 機能名
  - `icon` : 権限アイコン（Ⓦ=書き込み権限、Ⓡ=読み取り専用）
- `updatedAt` : 最終更新日時

エラー時（400 Bad Request）

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **認証チェック**
   - JWT認証でorganizationIdを取得
   - System AdminまたはData Adminの権限を確認
2. **データ取得**
   - organization_idでフィルタ（マルチテナント分離）
   - roleテーブルとrole_permissionテーブルを結合
   - 各ロールの権限情報を集約
3. **権限アイコン生成**
   - 書き込み権限あり: ⓦ
   - 読み取り専用: Ⓡ

バリデーション

必須チェック

- [ ] 認証トークンが有効
- [ ] organizationIdが取得できる

データベーステーブル

使用テーブル

1. **role**（主テーブル）
   - organization_id, role, role_name, initial_screen_function_name, is_active
   - create_user_id, create_at, update_user_id, update_at
2. **role_permission**（権限テーブル）
   - organization_id, role, function_name, function_write_permissions
   - create_user_id, create_at, update_user_id, update_at

SQL例

```
SELECT
  r.role,
  r.role_name,
  r.initial_screen_function_name,
  r.is_active,
  r.update_at,
  COALESCE(
    STRING_AGG(DISTINCT rp.function_name, ',' ORDER BY rp.function_name),
    ''
  ) as function_name
FROM role r
LEFT JOIN role_permission rp ON r.organization_id = rp.organization_id AND r.role = rp.role
WHERE r.organization_id = $1
GROUP BY r.role, r.role_name, r.initial_screen_function_name, r.is_active, r.update_at
ORDER BY r.role;
```

セキュリティ

マルチテナント分離

- [ ] organization_id フィルタ必須

- [ ] 自組織のロールのみ取得可能

認証・認可

- [ ] JWT認証必須
- [ ] System AdminまたはData Adminの権限が必要

入力検証

- [ ] SQLインジェクション対策（Parameterized Queries）

パフォーマンス

- インデックス: role (organization_id, is_active)
- インデックス: role_permission (organization_id, role)
- トランザクション: 不要（読み取り専用）
- キャッシュ: 不要（リアルタイム性重視）

---

2. ロールCSVダウンロードAPI

概要

ロール一覧と権限情報をCSV形式でダウンロードします。フィルタ条件を適用できます。

エンドポイント

```
GET /api/v1/master/role/download-csv
```

HTTPメソッド

GET

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.get('/role/download-csv',
  authMiddleware,
  roleController.downloadRoleCsv
);
```

リクエスト

クエリパラメータ

パラメータ	型	必須	デフォルト	説明
searchText	string	×	-	機能名で検索
roleNameFilter	string	×	-	ロール名でフィルタ

リクエスト例

```
GET /api/v1/master/role/download-csv?roleNameFilter=システム管理者
GET /api/v1/master/role/download-csv?searchText=management
```

レスポンス

成功時（200 OK）

Content-Type: `text/csv; charset=utf-8`

ファイル名: `role_YYYYMMDDhhmmss.csv`

CSV形式 (UTF-8 BOM付き):

```
role_cd,role_name,function_cd,permission,initial_screen_function_cd,is_active,update_at
system_admin,システム管理者,user_management,Ⓦ,system_overview,true,2026/2/13 10:00
system_admin,システム管理者,master_management,Ⓡ,system_overview,true,2026/2/13 10:00
data_admin,データ管理者,master_management,Ⓡ,master_management,true,2026/2/13 10:00
```

カラム定義:

- `role_cd` : ロールCD
- `role_name` : ロール名
- `function_cd` : 機能CD（各機能ごとに行を分ける）
- `permission` : 権限アイコン（Ⓦ=書き込み、Ⓡ=読み取り）
- `initial_screen_function_cd` : 初期画面機能CD
- `is_active` : アクティブ状態（true/false）
- `update_at` : 最終更新日時

重要な特徴:

- **1ロール = 複数行**: 各ロールが持つ機能ごとに行を分けて出力
- 例: system_adminロールがuser_managementとmaster_managementの2つの機能を持つ場合、2行出力される

ビジネスロジック

1. **データ取得**
   - getRoles()と同じロジックでデータ取得
   - organization_idでフィルタ
2. **フィルタリング**
   - roleNameFilter: 完全一致でフィルタ
   - searchText: 機能名に対して部分一致検索
3. **CSV生成**

- Papa.unparseを使用してCSV生成
- 各ロールの機能ごとに行を展開（flatMap）
- UTF-8 BOMを付与（Excel対応）

4. **ファイル名生成**
   - `role_YYYYMMDDhhmmss.csv` 形式
   - タイムスタンプを含める
5. **レスポンス**
   - Content-Typeヘッダーを設定
   - Content-Dispositionヘッダーでファイル名を指定

バリデーション

- [ ] 認証トークンが有効
- [ ] organizationIdが取得できる
- [ ] フィルタパラメータが有効（指定時）

データベーステーブル

- ロール一覧取得APIと同じ

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] JWT認証必須
- [ ] 適切な権限チェック

パフォーマンス

- 大量データの場合はストリーミング処理を検討
- インデックス活用でクエリ高速化

---

3. ロールCSVアップロードAPI

概要

CSV形式でロールデータと権限情報を一括登録・更新します。

エンドポイント

```
POST /api/v1/master/role/upload-csv
```

HTTPメソッド

POST

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.post('/role/upload-csv',
  authMiddleware,
  upload.single('file'),      // Multerミドルウェア
  roleController.uploadRoleCsv
);
```

リクエスト

マルチパートフォームデータ

Content-Type: `multipart/form-data`

フィールド	型	必須	説明
file	File	○	CSVファイル

CSV形式

```
role_cd,role_name,function_cd,permission,initial_screen_function_cd,is_active,update_at
system_admin,システム管理者,user_management,Ⓦ,system_overview,true,2026/2/13 10:00
system_admin,システム管理者,master_management,Ⓡ,system_overview,true,2026/2/13 10:00
data_admin,データ管理者,master_management,Ⓡ,master_management,true,2026/2/13 10:00
```

カラム定義:

- `role_cd` : ロールCD（必須、50文字以内）
- `role_name` : ロール名（必須、200文字以内）
- `function_cd` : 機能CD（各機能ごとに行を分ける）
- `permission` : 権限アイコン（Ⓦ or Ⓡ）
- `initial_screen_function_cd` : 初期画面機能CD
- `is_active` : アクティブ状態（必須、true/false）
- `update_at` : 最終更新日時（参考情報、更新処理には使用しない）

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "message": "ロールCSVのアップロードが完了しました"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（400 Bad Request）

```
{
  "success": false,
  "error": {
    "code": "BAD_REQUEST",
    "message": "CSVファイルがアップロードされていません"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **ファイル検証**
   - CSVファイルの存在確認
   - ファイルサイズチェック（最大10MB）
   - MIMEタイプチェック（text/csv）
2. **CSVパース**
   - Papa.parseを使用
   - ヘッダー行の検証
   - 各行のバリデーション
3. **データ処理**
   - トランザクション開始
   - ロールごとにグループ化
   - 既存データの確認（role_cd）
   - ロール情報の新規作成または更新
   - 権限情報の削除→再作成
   - エラー発生時はロールバック
4. **結果返却**

- 処理完了メッセージを返却

バリデーション

ファイルチェック

- [ ] ファイルが存在する
- [ ] ファイルサイズが10MB以下
- [ ] MIMEタイプがtext/csv

データチェック（各行）

- [ ] role_cdが存在する
- [ ] role_cdが50文字以内
- [ ] role_nameが存在する
- [ ] role_nameが200文字以内
- [ ] is_activeがboolean（true/false）
- [ ] function_cdが存在する（権限がある場合）
- [ ] permissionがⓌまたはⓇ（権限がある場合）

データベーステーブル

使用テーブル

1. **role**（主テーブル）
2. **role_permission**（権限テーブル）

SQL例

```
-- トランザクション開始
BEGIN;

-- 既存ロール確認
SELECT role FROM role
```

```
WHERE organization_id = $1 AND role = $2;

-- ロール作成/更新
INSERT INTO role (
  organization_id, role, role_name, initial_screen_function_name, is_active,
  create_user_id, create_at, update_user_id, update_at
) VALUES ($1, $2, $3, $4, $5, $6, NOW(), $6, NOW())
ON CONFLICT (organization_id, role) DO UPDATE SET
  role_name = EXCLUDED.role_name,
  initial_screen_function_name = EXCLUDED.initial_screen_function_name,
  is_active = EXCLUDED.is_active,
  update_user_id = EXCLUDED.update_user_id,
  update_at = NOW();

-- 既存権限削除
DELETE FROM role_permission
WHERE organization_id = $1 AND role = $2;

-- 権限作成
INSERT INTO role_permission (
  organization_id, role, function_name, function_write_permissions,
  create_user_id, create_at, update_user_id, update_at
) VALUES ($1, $2, $3, $4, $5, NOW(), $5, NOW());

COMMIT;
```

セキュリティ

マルチテナント分離

- [ ] organization_id フィルタ必須
- [ ] 自組織のデータのみ操作可能

ファイルアップロード対策

- [ ] ファイルサイズ制限（10MB）
- [ ] 許可されたMIMEタイプのみ
- [ ] ファイル内容のバリデーション

入力検証

- [ ] SQLインジェクション対策

- ☐ CSVインジェクション対策

パフォーマンス

- バッチ処理でINSERT/UPDATE
- トランザクション使用
- エラー時は全ロールバック

---

4. ロール詳細取得API

概要

ロールCDを指定して、ロールの詳細情報と権限情報を取得します。

エンドポイント

```
GET /api/v1/master/role/:roleId
```

HTTPメソッド

GET

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.get('/role/:roleId',
  authMiddleware,
```

```
  [
    param('roleId')
      .notEmpty()
      .withMessage('roleId is required')
      .isString()
      .withMessage('roleId must be a string'),
  ],
  handleValidationErrors,
  roleController.getRoleById
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明
roleId	string	○	ロールCD

リクエスト例

```
GET /api/v1/master/role/system_admin
```

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "roleId": "system_admin",
    "roleName": "システム管理者",
    "functionName": "user_management,master_management",
    "initialScreenFunctionName": "system_overview",
    "isActive": true,
    "functions": [
      {
        "name": "user_management",
        "icon": "Ⓦ"
```

```
      },
      {
        "name": "master_management",
        "icon": "Ⓡ"
      }
    ],
    "updatedAt": "2026-02-13T10:00:00Z"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
    "code": "ROLE_NOT_FOUND",
    "message": "Role not found"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **データ取得**
   - roleとorganization_idでロールを検索
   - role_permissionテーブルと結合して権限情報を取得
   - 該当データが存在しない場合は404エラー

バリデーション

- [ ] roleIdが空でない

- [ ] roleIdが文字列

データベーステーブル

SQL例

```
SELECT
  r.role,
  r.role_name,
  r.initial_screen_function_name,
  r.is_active,
  r.update_at,
  COALESCE(
    STRING_AGG(DISTINCT rp.function_name, ',' ORDER BY rp.function_name),
    ''
  ) as function_name
FROM role r
LEFT JOIN role_permission rp ON r.organization_id = rp.organization_id AND r.role = rp.role
WHERE r.organization_id = $1 AND r.role = $2
GROUP BY r.role, r.role_name, r.initial_screen_function_name, r.is_active, r.update_at;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] JWT認証必須

パフォーマンス

- プライマリキー検索（高速）
- トランザクション不要

---

5. ロール新規作成API

概要

新しいロールを作成します。

エンドポイント

```
POST /api/v1/master/role
```

HTTPメソッド

POST

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.post('/role',
  authMiddleware,
  [
    body('role')
      .notEmpty()
      .withMessage('ロールCDは必須です')
      .isString()
      .withMessage('ロールCDは文字列である必要があります'),
    body('roleName')
      .notEmpty()
      .withMessage('ロール名は必須です')
      .isString()
      .withMessage('ロール名は文字列である必要があります'),
    body('initialScreenFunctionName')
      .optional({ nullable: true })
      .isString()
      .withMessage('初期画面機能名は文字列である必要があります'),
    body('isActive')
      .optional()
      .isBoolean()
      .withMessage('isActiveはブール値である必要があります'),
  ],
```

```
  handleValidationErrors,
  roleController.createRole
);
```

リクエスト

リクエストボディ

```
{
  "role": "experiment_admin",
  "roleName": "実験管理者",
  "initialScreenFunctionName": "project_management",
  "isActive": true
}
```

パラメータ	型	必須	デフォルト	説明
role	string	○	-	ロールCD（50文字以内）
roleName	string	○	-	ロール名（200文字以内）
initialScreenFunctionName	string	×	null	初期画面機能名
isActive	boolean	×	true	アクティブ状態

レスポンス

成功時（201 Created）

```
{
  "success": true,
  "message": "Role created successfully",
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（409 Conflict）

```
{
  "success": false,
  "error": {
    "code": "DUPLICATE_ROLE",
    "message": "Role already exists"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **重複チェック**
   - 同一organization_id内で同じroleが存在しないか確認
2. **データ作成**
   - 新規レコードをINSERT
   - create_user_id, update_user_idに現在のユーザーIDを設定

バリデーション

- [ ] roleが必須
- [ ] roleが50文字以内
- [ ] roleNameが必須
- [ ] roleNameが200文字以内
- [ ] isActiveがboolean（指定時）

データベーステーブル

SQL例

```
-- 重複チェック
SELECT role FROM role
WHERE organization_id = $1 AND role = $2;

-- 新規作成
INSERT INTO role (
  organization_id, role, role_name, initial_screen_function_name, is_active,
  create_user_id, create_at, update_user_id, update_at
) VALUES ($1, $2, $3, $4, $5, $6, NOW(), $6, NOW())
RETURNING *;
```

セキュリティ

- [ ] organization_id必須
- [ ] SQLインジェクション対策

パフォーマンス

- トランザクション使用
- ユニーク制約でINSERT時の競合検知

---

6. ロール更新API

概要

既存のロール情報を更新します。

エンドポイント

```
PUT /api/v1/master/role/:roleId
```

HTTPメソッド

PUT

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.put('/role/:roleId',
  authMiddleware,
  [
    param('roleId')
      .notEmpty()
      .withMessage('roleId is required')
      .isString()
      .withMessage('roleId must be a string'),
    body('roleName')
      .optional()
      .isString()
      .withMessage('ロール名は文字列である必要があります'),
    body('initialScreenFunctionName')
      .optional({ nullable: true })
      .isString()
      .withMessage('初期画面機能名は文字列である必要があります'),
    body('isActive')
      .optional()
      .isBoolean()
      .withMessage('isActiveはブール値である必要があります'),
  ],
  handleValidationErrors,
  roleController.updateRole
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明
roleId	string	○	ロールCD

リクエストボディ

```
{
  "roleName": "実験管理者（更新）",
  "initialScreenFunctionName": "project_list",
  "isActive": true
}
```

パラメータ	型	必須	説明
roleName	string	×	ロール名（200文字以内）
initialScreenFunctionName	string	×	初期画面機能名
isActive	boolean	×	アクティブ状態

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "message": "Role updated successfully",
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
    "code": "ROLE_NOT_FOUND",
    "message": "Role not found"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **存在確認**
   - organization_idとroleでロールを検索
   - 存在しない場合は404エラー
2. **データ更新**
   - 指定されたフィールドのみ更新
   - update_user_id, update_atを更新

バリデーション

- [ ] roleIdが必須
- [ ] roleNameが200文字以内（指定時）
- [ ] isActiveがboolean（指定時）

データベーステーブル

SQL例

```
UPDATE role
SET
```

```
  role_name = COALESCE($3, role_name),
  initial_screen_function_name = COALESCE($4, initial_screen_function_name),
  is_active = COALESCE($5, is_active),
  update_user_id = $6,
  update_at = NOW()
WHERE organization_id = $1 AND role = $2
RETURNING *;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] SQLインジェクション対策

パフォーマンス

- プライマリキー更新（高速）
- トランザクション使用

---

7. ロール権限更新API ⭐

概要

ロールに紐づく機能権限を更新します。**ロール管理APIの特有機能**です。

エンドポイント

```
PUT /api/v1/master/role/:roleId/permissions
```

HTTPメソッド

PUT

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.put('/role/:roleId/permissions',
  authMiddleware,
  [
    param('roleId')
      .notEmpty()
      .withMessage('roleId is required')
      .isString()
      .withMessage('roleId must be a string'),
    body('functionName')
      .isString()
      .withMessage('functionName must be a string'),
    body('functionWritePermissions')
      .optional()
      .isString()
      .withMessage('functionWritePermissions must be a string'),
  ],
  handleValidationErrors,
  roleController.updateRolePermissions
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明
roleId	string	○	ロールCD

リクエストボディ

```
{
  "functionName": "user_management,master_management",
```

```
  "functionWritePermissions": "user_management"
}
```

パラメータ	型	必須	説明
functionName	string	○	機能名（カンマ区切りで複数指定可能）
functionWritePermissions	string	×	書き込み権限を持つ機能名（カンマ区切り）

権限の意味:

- `functionName` に含まれる: その機能へのアクセス権限あり
- `functionWritePermissions` に含まれる: 書き込み権限あり（Ⓦ）
- `functionName` にのみ含まれる: 読み取り専用（Ⓡ）

例:

```
{
  "functionName": "user_management,master_management,project_management",
  "functionWritePermissions": "user_management,master_management"
}
```

→ user_management: Ⓦ、master_management: Ⓦ、project_management: Ⓡ

全権限削除:

```
{
  "functionName": "",
  "functionWritePermissions": ""
}
```

レスポンス

成功時（200 OK）

```
{
  "success": true,
```

```
  "message": "Role permissions updated successfully",
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
    "code": "ROLE_NOT_FOUND",
    "message": "Role not found"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **ロール存在確認**
   - organization_idとroleでロールを検索
   - 存在しない場合は404エラー
2. **権限削除**
   - 該当ロールの既存権限を全削除
3. **権限再作成**
   - functionNameをカンマで分割
   - 各機能に対してrole_permissionレコードを作成
   - functionWritePermissionsに含まれる機能は書き込み権限付き
4. **更新日時更新**

- roleテーブルのupdate_atを更新

バリデーション

- [ ] roleIdが必須
- [ ] functionNameが文字列（空文字列も許容）
- [ ] functionWritePermissionsが文字列（指定時）

データベーステーブル

SQL例

```
-- トランザクション開始
BEGIN;

-- ロール存在確認
SELECT role FROM role
WHERE organization_id = $1 AND role = $2;

-- 既存権限削除
DELETE FROM role_permission
WHERE organization_id = $1 AND role = $2;

-- 権限作成（機能ごとにINSERT）
INSERT INTO role_permission (
  organization_id, role, function_name, function_write_permissions,
  create_user_id, create_at, update_user_id, update_at
) VALUES ($1, $2, $3, $4, $5, NOW(), $5, NOW());

-- ロールの更新日時を更新
UPDATE role
SET update_user_id = $3, update_at = NOW()
WHERE organization_id = $1 AND role = $2;

COMMIT;
```

セキュリティ

- [ ] organization_id フィルタ必須

- ☐ SQLインジェクション対策
- ☐ トランザクションで整合性を保証

パフォーマンス

- トランザクション使用
- 既存権限を全削除→再作成（シンプルで安全）

---

8. ロールアクティブ状態更新API

概要

ロールのアクティブ状態のみを更新します。

エンドポイント

```
PATCH /api/v1/master/role/:roleId/status
```

HTTPメソッド

PATCH

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.patch('/role/:roleId/status',
  authMiddleware,
```

```
  [
    param('roleId')
      .notEmpty()
      .withMessage('roleId is required')
      .isString()
      .withMessage('roleId must be a string'),
    body('isActive')
      .notEmpty()
      .withMessage('isActive is required')
      .isBoolean()
      .withMessage('isActive must be a boolean'),
  ],
  handleValidationErrors,
  roleController.updateRoleStatus
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明
roleId	string	○	ロールCD

リクエストボディ

```
{
  "isActive": false
}
```

パラメータ	型	必須	説明
isActive	boolean	○	アクティブ状態

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "message": "Role status updated successfully",
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
    "code": "ROLE_NOT_FOUND",
    "message": "Role not found"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **存在確認**
    - organization_idとroleでロールを検索
2. **ステータス更新**
    - is_activeフィールドのみ更新
    - update_user_id, update_atを更新

バリデーション

- [ ] roleIdが必須

- [ ] isActiveが必須
- [ ] isActiveがboolean

データベーステーブル

SQL例

```
UPDATE role
SET
  is_active = $3,
  update_user_id = $4,
  update_at = NOW()
WHERE organization_id = $1 AND role = $2
RETURNING *;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] SQLインジェクション対策

パフォーマンス

- プライマリキー更新（高速）
- トランザクション使用

🔗 関連ドキュメント

API設計書

- api_list.md - API一覧対応表
- api_master_management.md - マスタ管理API
- api_department_management.md - 部署管理API

- api_corporation_management.md - 会社管理API

規約・ガイドライン

- API設計ポリシー - API設計の統一ルール
- バックエンド規約 - 実装規約
- セキュリティ規約 - セキュリティ要件

参照ドキュメント

- データベース構造 - DB設計
- API構成定義 - API構成の参照ルール

📊 実装ファイルパス

ルートファイル

```
backend/src/routes/roleRoutes.ts
```

コントローラーファイル

```
backend/src/controllers/roleController.ts
```

サービスファイル

```
backend/src/services/roleService.ts
```

リポジトリファイル

```
backend/src/repositories/roleRepository.ts
```

フロントエンドサービス

```
frontend/src/services/roleApiService.ts
```

フロントエンドコンポーネント

```
frontend/src/components/SystemManagement/RoleManagement.tsx
```

✅ セキュリティチェックリスト

実装時に以下を確認してください：

認証・認可

- [ ] すべてのエンドポイントでJWT認証が必須
- [ ] authMiddlewareが適用されている
- [ ] System AdminまたはData Adminの権限が必要

マルチテナント分離

- [ ] すべてのDBクエリにorganization_idフィルタが含まれる
- [ ] クロステナントアクセスが不可能
- [ ] authMiddlewareでorganizationIdを取得

入力検証

- [ ] すべてのパラメータがバリデーションされている
- [ ] Parameterized Queriesを使用（SQLインジェクション対策）
- [ ] XSS対策（入力サニタイゼーション）

- [ ] 文字列長制限が強制されている

CSVアップロード対策

- [ ] ファイルサイズ制限（10MB）
- [ ] 許可されたMIMEタイプのみ（text/csv）
- [ ] ファイル内容のバリデーション
- [ ] CSVインジェクション対策

エラーハンドリング

- [ ] 統一されたエラーレスポンス形式
- [ ] エラーメッセージに機密情報が含まれない
- [ ] 適切なHTTPステータスコードを返す

ログ記録

- [ ] 重要な操作の監査証跡を記録
- [ ] エラー発生時の詳細ログ

📝 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-13	初版作成	Cline
		- ロール管理API 8エンドポイントの設計書作成	
		- 標準CRUD操作 + 権限更新機能	
		- 実装済み（100%）	

注意: この設計書は実装済みAPIの仕様を文書化したものです。実装の詳細については、実際のソースコード（backend/src/routes/roleRoutes.ts等）を参照してください。